RProgramming: Pseudocode Writing Task (CO1-Assessment Tool 2)

Name: Sure Jaswanth Venkata Rahul

REG NO: 192425048

Question 1: Variables and Data Types Analysis

Problem understanding. Input: values of different types. Output: the type of each variable plus arithmetic and logical results. Constraint: arithmetic needs numeric operands; logical operators work on logical values.

Pseudocode

```
START
  DECLARE num_var   <- 25.5          (numeric)
  DECLARE int_var   <- 10            (integer)
  DECLARE char_var  <- "R Programming" (character)
  DECLARE flag_a    <- TRUE          (logical)
  DECLARE flag_b    <- FALSE         (logical)
  DISPLAY the data type of each variable
  COMPUTE sum, difference, product, quotient, modulus, power of num_var and int_var
  COMPUTE comparison results (>, <, ==) on num_var and int_var
  COMPUTE AND, OR, NOT on flag_a and flag_b
  DISPLAY char_var and its length
  DISPLAY all results
END
```

R code

```
num_var     <- 25.5
int_var     <- 10L
char_var    <- "R Programming"
logical_var <- TRUE
flag_b      <- FALSE
cat("Type of num_var     :", class(num_var), "\n")
cat("Type of int_var     :", class(int_var), "\n")
cat("Type of char_var    :", class(char_var), "\n")
cat("Type of logical_var :", class(logical_var), "\n\n")
cat("Addition       :", num_var + int_var, "\n")
cat("Subtraction    :", num_var - int_var, "\n")
cat("Multiplication :", num_var * int_var, "\n")
cat("Division       :", num_var / int_var, "\n")
cat("Modulus        :", num_var %% int_var, "\n")
cat("Power          :", num_var ^ 2, "\n\n")


cat("num_var > int_var  :", num_var > int_var, "\n")
cat("num_var == int_var :", num_var == int_var, "\n\n")
```

```
cat("TRUE AND FALSE :", logical_var & flag_b, "\n")
cat("TRUE OR FALSE  :", logical_var | flag_b, "\n")
cat("NOT TRUE       :", !logical_var, "\n\n")
cat("String         :", char_var, "\n")
cat("Length (chars):", nchar(char_var), "\n")
cat("Upper case     :", toupper(char_var), "\n")
```

Question 2: Data Structures Implementation

Problem understanding. Input: sample data for a vector, a list and a matrix. Output: results of indexing, updating and aggregation, followed by a comparison of the three structures.

Pseudocode

```
START
  CREATE vector  v <- (10, 20, 30, 40, 50)
  CREATE list    l <- (name, age, marks vector, passed flag)
  CREATE matrix  m <- 3x3 matrix of numbers 1..9

  INDEXING:
    DISPLAY v[2], v[2..4]
    DISPLAY l["name"], l[["marks"]][1]
    DISPLAY m[2,3], row 1, column 2

  UPDATING:
    SET v[3] <- 99
    SET l["age"] <- 21 and ADD new element city
    SET m[1,1] <- 100

  AGGREGATION:
    COMPUTE sum, mean, max of v
    COMPUTE mean of l.marks
    COMPUTE row sums, column sums of m

  DISPLAY comparison of the structures
END
```

R code

```
v <- c(10, 20, 30, 40, 50)

l <- list(name = "Arun", age = 20, marks = c(78, 85, 92), passed = TRUE)

m <- matrix(1:9, nrow = 3, ncol = 3, byrow = TRUE)

cat("v[2]       :", v[2], "\n")
cat("v[2:4]     :", v[2:4], "\n")
cat("l$name     :", l$name, "\n")
cat("l$marks[1] :", l[["marks"]][1], "\n")
cat("m[2,3]     :", m[2, 3], "\n")
cat("Row 1 of m :", m[1, ], "\n")
cat("Col 2 of m :", m[, 2], "\n\n")
v[3]     <- 99
l$age    <- 21
```

```
l$city   <- "Chennai"      # add a new element
m[1, 1]  <- 100

print(v)
str(l)
print(m)
cat("Vector sum  :", sum(v), "\n")
cat("Vector mean :", mean(v), "\n")
cat("Vector max  :", max(v), "\n")
cat("Mean marks  :", mean(l$marks), "\n")
cat("Row sums    :", rowSums(m), "\n")
cat("Column sums :", colSums(m), "\n")
cat("\nvector is.atomic:", is.atomic(v), "| list is.atomic:", is.atomic(l), "\n")
cat("dim(m):", dim(m), "| length(l):", length(l), "| length(v):", length(v), "\n")
```

Question 3: Matrix and Array Operations

Problem understanding. Input: two 3x3 matrices and two 2x3x2 arrays. Output: sum, matrix product and element-wise product. Constraint: matrix multiplication needs columns of A equal to rows of B, while addition and element-wise operations need identical dimensions.

Pseudocode

```
START
  CREATE matrix A (3x3) with values 1..9 row-wise
  CREATE matrix B (3x3) with values 9..1 row-wise
  CREATE 3D arrays X, Y with dim (2 rows, 3 columns, 2 layers)

  IF dimensions of A and B are equal THEN
     C <- A + B                  (addition)
     D <- A * B                  (element-wise)
  IF columns(A) = rows(B) THEN
     E <- A x B                  (matrix multiplication)

  Z1 <- X + Y,  Z2 <- X * Y        (array operations)
  DISPLAY all results
  ANALYZE the difference between A * B and A %*% B
END
```

R code

```
A <- matrix(1:9, nrow = 3, byrow = TRUE)
B <- matrix(9:1, nrow = 3, byrow = TRUE)
X <- array(1:12, dim = c(2, 3, 2))
Y <- array(12:1, dim = c(2, 3, 2))

cat("Matrix A:\n"); print(A)
cat("Matrix B:\n"); print(B)
if (all(dim(A) == dim(B))) {
  cat("\nA + B (addition):\n");        print(A + B)
  cat("\nA * B (element-wise):\n");     print(A * B)
}

if (ncol(A) == nrow(B)) {
```

```
  cat("\nA %*% B (matrix multiplication):\n")
  print(A %*% B)
}

cat("\nTranspose of A:\n"); print(t(A))
cat("\nArray X:\n"); print(X)
cat("\nX + Y:\n");   print(X + Y)
cat("\nX * Y (element-wise):\n"); print(X * Y)
cat("\nSum of each layer of X:", apply(X, 3, sum), "\n")
cat("Dimensions of X:", dim(X), "\n")
```

Question 4: Data Frame Construction and Manipulation

Problem understanding. Input: student records with numeric and character fields. Output: a data frame that is modified (column added, deleted, changed) and summarised.

Pseudocode

```
START
  CREATE data frame students WITH columns:
     ID (numeric), Name (character), Dept (character),
     Marks (numeric), Age (numeric)
  DISPLAY data frame

  ADD column Grade:
     FOR each student
        IF Marks >= 90 THEN Grade <- "A"
        ELSE IF Marks >= 75 THEN Grade <- "B"
        ELSE IF Marks >= 50 THEN Grade <- "C"
        ELSE Grade <- "F"
  MODIFY column Marks: add 5 bonus marks to every student (cap at 100)
  DELETE column Age

  SUMMARY:
     DISPLAY structure, dimensions, column names
     COMPUTE mean, max, min of Marks
     COUNT students per Dept
     COMPUTE average Marks per Dept
END
```

R code

```
students <- data.frame(
  ID    = c(101, 102, 103, 104, 105),
  Name  = c("Arun", "Bala", "Charu", "Divya", "Elan"),
  Dept  = c("IT", "CSE", "IT", "ECE", "CSE"),
  Marks = c(88, 92, 67, 45, 79),
  Age   = c(19, 20, 19, 21, 20),
  stringsAsFactors = FALSE
)
cat("Original data frame:\n"); print(students)
students$Grade <- ifelse(students$Marks >= 90, "A",
                  ifelse(students$Marks >= 75, "B",
                  ifelse(students$Marks >= 50, "C", "F")))
```

```
students$Marks <- pmin(students$Marks + 5, 100)
students$Age <- NULL

cat("\nAfter add / modify / delete:\n"); print(students)
cat("\nDimensions:", dim(students), "\n")
cat("Column names:", names(students), "\n")
str(students)
summary(students$Marks)
cat("\nMean Marks:", mean(students$Marks), "\n")
cat("Highest Marks:", max(students$Marks), "\n")
cat("Lowest Marks :", min(students$Marks), "\n")
cat("\nStudents per department:\n"); print(table(students$Dept))
cat("\nAverage marks per department:\n")
print(aggregate(Marks ~ Dept, data = students, FUN = mean))
cat("\nStudents with Marks > 80, sorted descending:\n")
top <- students[students$Marks > 80, ]
print(top[order(-top$Marks), ])
```

Question 5: Functions and Logical Operations

Problem understanding. Input: a vector of marks and a pass mark (parameters). Output: logical vector results and computed summary values returned from a function. Edge cases: empty input and missing values.

Pseudocode

```
FUNCTION analyze_marks(marks, pass_mark = 40)
   IF length of marks = 0 THEN
      RETURN message "No data"
   passed      <- marks >= pass_mark        (logical vector)
   failed      <- NOT passed
   distinction <- marks >= 75 AND passed
   pass_count  <- COUNT of TRUE in passed
   pass_percent <- pass_count / length(marks) * 100
   all_passed  <- ALL(passed)
   any_failed  <- ANY(failed)
   positions   <- indexes where failed is TRUE
   RETURN list(passed, failed, distinction, pass_count,
               pass_percent, all_passed, any_failed, positions)
END FUNCTION

MAIN
   marks <- (78, 35, 92, 40, 55, 28, 81)
   result <- analyze_marks(marks, 40)
   DISPLAY every item of result
END
```

R code

```
analyze_marks <- function(marks, pass_mark = 40) {

  # Handle edge cases
  if (length(marks) == 0) {
    return("No data provided")
```

```
  }
  passed      <- marks >= pass_mark
  failed      <- !passed
  distinction <- marks >= 75 & passed

  pass_count   <- sum(passed, na.rm = TRUE)
  pass_percent <- round(pass_count / length(marks) * 100, 2)
  return(list(
    passed       = passed,
    failed       = failed,
    distinction  = distinction,
    pass_count   = pass_count,
    pass_percent = pass_percent,
    all_passed   = all(passed),
    any_failed   = any(failed),
    fail_index   = which(failed)
  ))
}
marks  <- c(78, 35, 92, 40, 55, 28, 81)
result <- analyze_marks(marks, pass_mark = 40)

cat("Marks               :", marks, "\n")
cat("Passed (logical)    :", result$passed, "\n")
cat("Failed (logical)    :", result$failed, "\n")
cat("Distinction (logical):", result$distinction, "\n")
cat("Number passed       :", result$pass_count, "\n")
cat("Pass percentage     :", result$pass_percent, "%\n")
cat("All passed?         :", result$all_passed, "\n")
cat("Any failed?         :", result$any_failed, "\n")
cat("Failed positions    :", result$fail_index, "\n")
print(analyze_marks(c(60, 70, 80))$pass_percent)
print(analyze_marks(numeric(0)))
```

Code of Conduct:

I Sure Jaswanth Venkata Rahul (192425048)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

S Jaswanth

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	**20%**	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	**30%**	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	**20%**	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	**10%**	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	**20%**	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient